Optimal Military Troupe Stationing on a Tree

Design and Analysis of Algorithms Project Report

Ramdas Singh (bmat2423) Sathvik H S (bmat2428)

1 Problem Statement

Country XYZ has n important towns connected by roads. There are exactly $n-1$ roads, and every town is reachable from every other town by a unique sequence of roads. Two towns are called neighbors if a direct road connects them. The king decides to station military troupes in the towns. Each town can be assigned 0, 1, or 2 troupes, subject to the condition that every town assigned 0 troupes has at least one neighboring town assigned 2 troupes. Design an algorithm to assign troupes to towns so that the total number of troupes assigned is minimized.

A connected graph on n vertices with exactly $n-1$ edges, in which every pair of vertices is joined by a unique path, is precisely a *tree*. The towns and roads therefore form a tree $T = (V, E)$ with $|V| = n$.

A *troupe assignment*, or simply an *assignment*, is a map $f : V \to \{0, 1, 2\}$. An assignment f is *feasible* if, for every town v with $f(v) = 0$, at least one neighbor u of v has $f(u) = 2$.

The task is to compute a feasible assignment f that minimizes the total $|f| := \sum_{v \in V} f(v)$ and to report the minimum. We say that a 0-town is *covered* by any neighbor carrying 2 troupes. Feasibility therefore requires every 0-town to be covered. A feasible assignment always exists because $f \equiv 1$ has no 0-towns and costs n, so the optimum is at most n.

2 The Model: A Rooted-Tree Dynamic Program

The assignment condition makes a town dependent only on itself and its *neighbors*. This locality can be exploited using bottom-up dynamic programming on a rooted tree. We root the tree at an arbitrary town r and process it from the leaves upward. For a vertex v, let T_v be the subtree rooted at v, and let $C(v)$ be the set of children of v. The subtrees $\{T_u : u \in C(v)\}$ are pairwise vertex-disjoint, and $V(T_v) = \{v\} \cup \bigcup_{u \in C(v)} V(T_u)$. Crucially, the only edge joining T_u to the rest of T_v is the edge vu. Hence, once $f(v)$ is fixed, the assignment on each child subtree can be chosen independently. The assignment condition at any vertex $x \neq v$ in T_v depends only on the assignment within the child subtree containing x, with one exception. A child u may be covered by its parent v when $f(v) = 2$ and $f(u) = 0$. This is the optimal-substructure property that drives the recurrence.

The subtlety is that whether a vertex is covered may depend on a vertex *outside* its own subtree, namely its parent. We therefore track enough information at each vertex to decouple its subtree from the parent's eventual choice. Specifically, we maintain four quantities per vertex.

Definition 1 (Internal satisfaction)**.** Fix a rooted subtree T_v and an assignment f on $V(T_v)$. A vertex $x \in V(T_v)$ is *internally satisfied* if $f(x) \geq 1$, or if $f(x) = 0$ and some neighbor y of x *lying in* T_v has $f(y) = 2$. Every vertex of T_v other than v itself has all its neighbors inside T_v, so internal satisfaction for such a vertex coincides with the assignment condition.

Definition 2 (The four states)**.** For a vertex v, define the following minima over assignments f on $V(T_v)$, with the convention $\min \varnothing = +\infty$:

- $\mathrm{w}_1(v)$: the minimum of $|f|$ subject to $f(v) = 1$ and every vertex of T_v being internally satisfied.

- $\mathrm{w}_2(v)$: the minimum of $|f|$ subject to $f(v) = 2$ and every vertex of T_v being internally satisfied.
- $\mathrm{OPT}(v)$: the minimum of $|f|$ subject to every vertex of T_v being internally satisfied, with $f(v)$ unrestricted.
- $\mathrm{DC}(v)$ ("don't care"): the minimum of $|f|$ subject to every vertex of T_v, except possibly v, being internally satisfied.

In words, $\mathrm{OPT}(v)$ is the cost of an optimal self-contained solution on T_v, while $\mathrm{DC}(v)$ is the cost when v is allowed to be an uncovered 0-town because its parent will provide the covering 2. We also write $\mathrm{w}_0(v)$ for the variant of $\mathrm{OPT}(v)$ in which $f(v) = 0$ is forced, so that $\mathrm{OPT}(v) = \min\{\mathrm{w}_0(v), \mathrm{w}_1(v), \mathrm{w}_2(v)\}$.

Because the root r has no parent, every town's assignment condition must be satisfied inside $T_r = T$. Therefore, the answer to the problem is exactly $\mathrm{OPT}(r)$.

2.1 The recurrences

Writing $S(v) = \sum_{u \in C(v)} \mathrm{OPT}(u)$ for the total optimum over the child subtrees, the four quantities satisfy

$$\mathrm{w}_1(v) = S(v) + 1, \tag{1}$$

$$\mathrm{w}_2(v) = \sum_{u \in C(v)} \mathrm{DC}(u) \; + \; 2, \tag{2}$$

$$\mathrm{w}_0(v) = \min_{w \in C(v)} \Big(\mathrm{w}_2(w) + \sum_{\substack{u \in C(v) \\ u \neq w}} \mathrm{OPT}(u) \Big) \; = \; \min_{w \in C(v)} \big(S(v) - \mathrm{OPT}(w) + \mathrm{w}_2(w) \big), \tag{3}$$

$$\mathrm{OPT}(v) = \min \big\{ \; \mathrm{w}_1(v), \; \mathrm{w}_2(v), \; \mathrm{w}_0(v) \; \big\}, \tag{4}$$

$$\mathrm{DC}(v) = \min \big\{ \; \mathrm{OPT}(v), \; S(v) \; \big\}. \tag{5}$$

A leaf has $C(v) = \varnothing$, so $S(v) = 0$. The base cases are $\mathrm{w}_1(v) = 1$, $\mathrm{w}_2(v) = 2$, and $\mathrm{w}_0(v) = +\infty$, giving $\mathrm{OPT}(v) = 1$ and $\mathrm{DC}(v) = 0$.

2.2 The algorithm

A single post-order traversal evaluates the arrays bottom-up. The solution to our problem is $\mathrm{OPT}[r]$.

Algorithm 1 Minimum troupe assignment on a tree

```
function MinTroupes(tree T, root r)
    for v in a post-order of T rooted at r do
        S ← Σ_{u∈C(v)} OPT[u]
        w1[v] ← S + 1
        w2[v] ← Σ_{u∈C(v)} DC[u] + 2
        w0 ← min_{w∈C(v)} (S − OPT[w] + w2[w])          ▷ +∞ if C(v) = ∅
        OPT[v] ← min{w1[v], w2[v], w0}
        DC[v] ← min{OPT[v], S}
    end for
    return OPT[r]
end function
```

The three sums and minima at v can be computed in a single pass over $C(v)$. To recover an optimal *assignment*, rather than merely its cost, store at each vertex which of the three cases

attained the minimum in (4), along with the covering child w for the 0-case. Also store which case attained the minimum in (5). A second, top-down pass then determines $f(v)$ for every vertex v, expanding each child in the mode (OPT, DC, or "must be 2") dictated by the stored choice.

3 Proof of Correctness

We prove that the recurrences correctly compute the four states. The correctness of the algorithm then follows because the answer is $\mathrm{OPT}(r)$. Throughout the proof, we use the structural fact that once $f(v)$ is fixed, the restrictions $f_u := f \restriction_{T_u}$ to the child subtrees are independent, and the only cross-subtree adjacency is given by each edge vu.

Lemma 3. $\mathrm{w}_1(v) = S(v) + 1$.

Proof. Let f be feasible for $\mathrm{w}_1(v)$, so that $f(v) = 1$ and every vertex of T_v is internally satisfied. Since $f(v) = 1 \neq 2$, the vertex v provides no cover. Therefore, each child u and every other vertex of T_u, whose neighbors all lie in T_u, must be internally satisfied within T_u. Thus, each f_u is feasible for $\mathrm{OPT}(u)$, giving $|f| = 1 + \sum_u |f_u| \geq 1 + S(v)$. Conversely, taking an $\mathrm{OPT}(u)$-optimal assignment f_u on each child subtree and setting $f(v) = 1$ yields a feasible assignment of cost exactly $1 + S(v)$. The vertex v is satisfied because $f(v) = 1$, and every other vertex retains its in-subtree witness. Hence, equality holds. ∎

Lemma 4. $\mathrm{w}_2(v) = \sum_{u \in C(v)} \mathrm{DC}(u) + 2$.

Proof. Let f be feasible for $\mathrm{w}_2(v)$, so that $f(v) = 2$ and every vertex of T_v is internally satisfied. For a child u with $f(u) = 0$, the covering 2 may now be v itself, so u need not be internally satisfied within T_u. Every other vertex of T_u must still be internally satisfied. Thus, each f_u is feasible for $\mathrm{DC}(u)$, and $|f| \geq 2 + \sum_u \mathrm{DC}(u)$. Conversely, choose a $\mathrm{DC}(u)$-optimal assignment f_u on each child subtree and set $f(v) = 2$. Then v is satisfied, any child u that was an uncovered 0-town under DC is now covered by v, and all other vertices retain their witnesses. Hence, f is feasible with cost $2 + \sum_u \mathrm{DC}(u)$, proving equality. ∎

Lemma 5. $\mathrm{w}_0(v) = \min_{w \in C(v)} \big(S(v) - \mathrm{OPT}(w) + \mathrm{w}_2(w)\big)$, *which is* $+\infty$ *when* $C(v) = \varnothing$.

Proof. Let f be feasible for $\mathrm{w}_0(v)$, so that $f(v) = 0$ and every vertex of T_v is internally satisfied. Since $f(v) = 0$, the vertex v must be covered from within T_v. Its only in-subtree neighbors are its children, so some child w must have $f(w) = 2$. For that child, f_w has $f(w) = 2$ and every vertex of T_w is internally satisfied because the neighbors of w other than v lie in T_w, and v contributes 0. Thus, f_w is feasible for $\mathrm{w}_2(w)$. For every other child $u \neq w$, the value $f(v) = 0$ provides no cover, so f_u must be feasible for $\mathrm{OPT}(u)$. Hence, $|f| \geq \mathrm{w}_2(w) + \sum_{u \neq w} \mathrm{OPT}(u) = S(v) - \mathrm{OPT}(w) + \mathrm{w}_2(w)$. Minimizing over the designated child w gives the lower bound.

Conversely, choose any child w, an optimal assignment for $\mathrm{w}_2(w)$ on T_w, and an optimal assignment for $\mathrm{OPT}(u)$ on every other child subtree. Setting $f(v) = 0$ gives a feasible assignment because v is covered by w. This assignment attains the stated bound, proving the reverse inequality. If v has no children, it cannot be covered, so no feasible assignment exists and $\mathrm{w}_0(v) = +\infty$. ∎

Lemma 6. $\mathrm{OPT}(v) = \min\{\mathrm{w}_0(v), \mathrm{w}_1(v), \mathrm{w}_2(v)\}$ *and* $\mathrm{DC}(v) = \min\{\mathrm{OPT}(v),\, S(v)\}$.

Proof. The first identity is immediate. A feasible assignment f for $\mathrm{OPT}(v)$ has $f(v) \in \{0, 1, 2\}$, and conditioning on that value gives exactly the three constrained minima.

For $\mathrm{DC}(v)$, the vertex v is exempt from the internal satisfaction requirement. If $f(v) \geq 1$, or if $f(v) = 0$ and has an in-subtree covering child, then every vertex of T_v is internally satisfied, and the cheapest such assignment costs $\mathrm{OPT}(v)$. The only remaining possibility is $f(v) = 0$ with no

covering child, in which case v relies on its parent. Since v provides no cover to its children, each child subtree must be feasible for $\mathrm{OPT}(u)$. The cheapest such assignment costs $\sum_u \mathrm{OPT}(u) = S(v)$. Taking the minimum of these two possibilities yields $\mathrm{DC}(v) = \min\{\mathrm{OPT}(v), S(v)\}$. ■

Theorem 7. MinTroupes(T, r) *returns the minimum number of troupes over all feasible assignments of* T.

Proof. We show by induction on $|V(T_v)|$ that the computed values $\mathrm{OPT}[v], \mathrm{DC}[v], \mathrm{w}_1[v]$, and $\mathrm{w}_2[v]$ equal the quantities defined in Definition 2. *Base case:* A leaf v has $C(v) = \varnothing$. The recurrences give $\mathrm{w}_1[v] = 1$, $\mathrm{w}_2[v] = 2$, $\mathrm{w}_0 = +\infty$, $\mathrm{OPT}[v] = 1$, and $\mathrm{DC}[v] = 0$. These values match the definitions directly. A lone 0-town cannot be covered, a lone 1-town is the cheapest internally satisfied assignment, and exempting the leaf allows the zero-cost assignment $f(v) = 0$. *Inductive step:* For an internal vertex v, every child u has a strictly smaller subtree. By the inductive hypothesis, $\mathrm{OPT}[u]$ and $\mathrm{DC}[u]$ are correct. Lemmas 3, 4, 5, and 6 then show that the values computed at v from (1)–(5) equal the defined quantities. Finally, since the root r has no parent, internal satisfaction throughout $T_r = T$ is exactly the feasibility condition for the whole tree. Hence, $\mathrm{OPT}(r)$ is the minimum cost of a feasible assignment, which is what the algorithm returns. ■

Corollary 8. *The top-down reconstruction outputs a feasible assignment* f *with* $|f| = \mathrm{OPT}(r)$.

Proof. Each stored choice records a case of the corresponding recurrence that attained the minimum for a vertex processed in a given mode. Expanding the children in the modes prescribed by that case, as established in Lemmas 3–6, reproduces an assignment achieving the optimal value at each level. The proofs of those lemmas show that each such assignment is feasible for its corresponding state. By induction, the assignment produced at the root is feasible and has cost $\mathrm{OPT}(r)$. ■

4 Complexity Analysis

Each vertex v is processed once, with work proportional to $1 + |C(v)|$. The sums S and w_2, along with the minimum for w_0, can be computed in a single scan of the children. Summing over all vertices,

$$\sum_{v \in V} \big(1 + |C(v)|\big) = n + \sum_{v \in V} |C(v)| = n + (n - 1) = \mathcal{O}(n),$$

since every non-root vertex is the child of exactly one parent, so $\sum_v |C(v)| = |E| = n - 1$. Building the rooted structure using BFS or DFS to determine the parents and post-order is also $\mathcal{O}(n)$. The four arrays and the explicit stack use $\mathcal{O}(n)$ space. Reconstruction adds another $\mathcal{O}(n)$ pass. Hence, the algorithm runs in $\mathcal{O}(n)$ time and uses $\mathcal{O}(n)$ space. This is optimal up to constant factors because merely reading the tree takes $\Omega(n)$ time.

5 Implementation

The algorithm is implemented in Python as the module `troupes.py`. It provides the linear-time dynamic program, the top-down reconstruction of an optimal assignment, a brute-force reference solver, and a verification suite that cross-checks the two over all labelled trees on up to six towns and over random trees for larger sizes. The complete source, together with the development notebook and this report, is available at

`https://github.com/MaxMight01/DAA_Project`.

The accompanying `README.md` lists the requirements and gives the commands for running the implementation and its verification suite.

6 Conclusion

Modelling the towns as a tree and rooting it reduces the problem to a four-state dynamic program (OPT, DC, w_1, w_2) evaluated in a single post-order pass. The states precisely encode the one piece of non-local information required by the assignment condition, namely whether a 0-town may rely on its parent for coverage. The resulting algorithm is provably correct by induction over subtrees and achieves optimal $\mathcal{O}(n)$ time and space complexity.